Alikable Design

Oct 13, 2026 · @Group 1

The big picture

Alikable builds a front end for a backend that already exists. The user gives their API endpoints, and a team of AI agents makes the web pages that use them.

Who it is for: backend engineers who can build APIs but find it hard to build a UI.

The problem: a backend with no UI is hard to demo, test, or share. Building the UI by hand takes days of front-end work.

The idea in one line: "Here are my endpoints, give me the UI."

The project uses CrewAI (4 crews, 7 agents) and is hosted on Hugging Face Spaces.

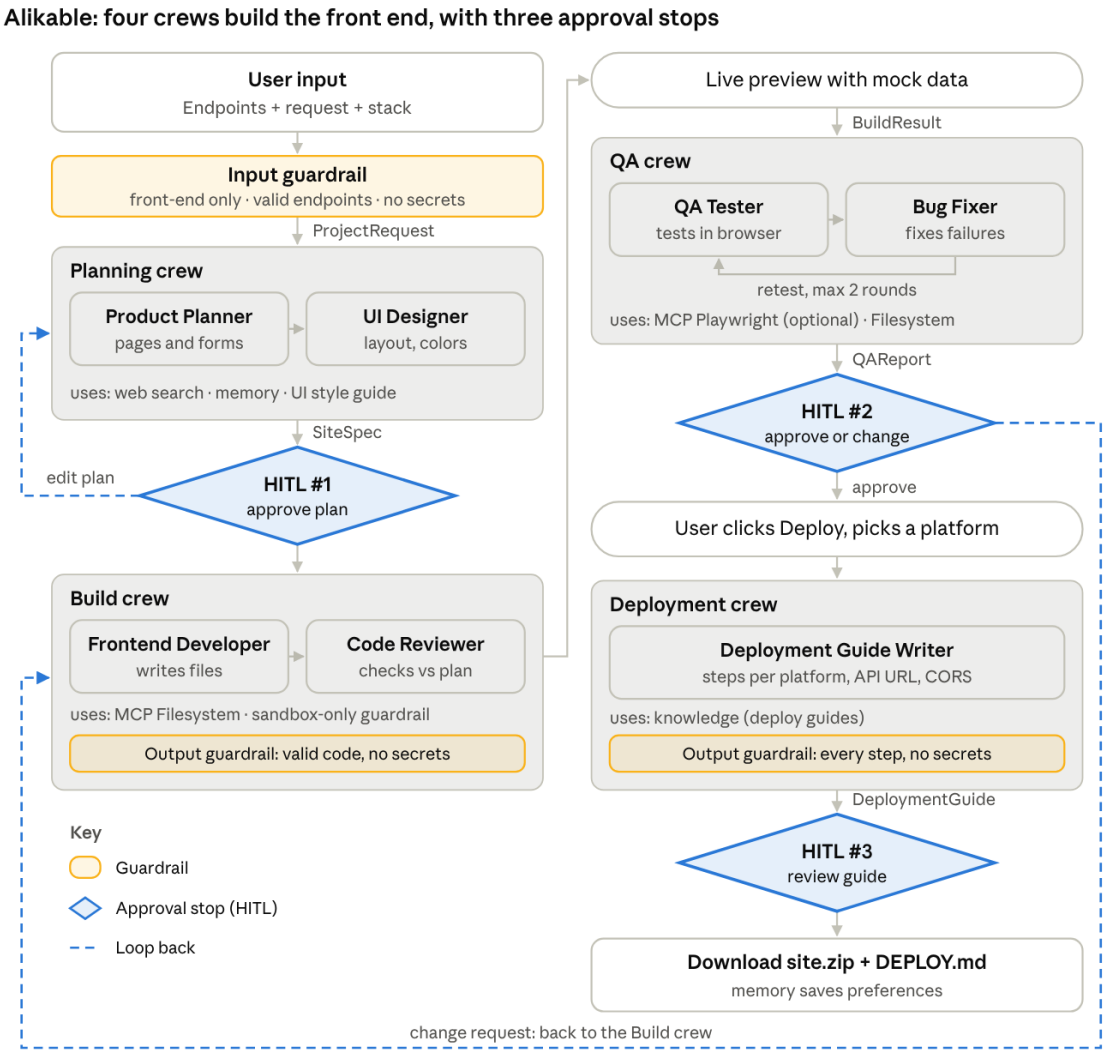
What the app will and will not do

The app makes only the front end. The backend is the user's, and deploying is the user's job.

The app WILL	The app will NOT
Take the user's endpoints (an OpenAPI file, or a pasted list) and a short description	Build or change the backend
Make pages, forms and buttons that call those endpoints, including a login page	Deploy the user's site
Show a live preview with fake (mock) data	Connect to the user's real backend
Stop and ask the user to approve at 3 points	Support every framework (HTML/CSS/JS first; React and Vue planned)

The app WILL	The app will NOT
Test the pages and fix bugs (at most 2 rounds)	
Give a download plus step-by-step deploy instructions	

The flow



System flow · 4 crews, 7 agents, 3 approval stops

The main path runs top to bottom. Dashed lines on the left are the edit loops; dashed boxes on the right are what each crew uses.

1. The user gives their endpoints (OpenAPI file or a list), a request, and a stack.
2. The input guardrail checks it and removes secrets.
3. The input is turned into `ProjectRequest`.
4. **Planning crew:** the Product Planner decides the pages and forms; the UI Designer decides the look. Output: `SiteSpec`.
5. **HITL #1:** the user approves the plan, or edits it and Planning runs again.
6. **Build crew:** the Frontend Developer writes the files into the sandbox; the Code Reviewer checks them against the plan. Output: `BuildResult`.
7. The live preview shows the pages with mock data.
8. **QA crew:** the QA Tester tests the pages; the Bug Fixer fixes failures, at most 2 rounds. Output: `QAReport`.
9. **HITL #2:** the user approves, or types a change that goes back to the Build crew.
10. The user clicks Deploy and picks a platform.
11. **Deployment crew:** the Deployment Guide Writer writes the steps. Output: `DeploymentGuide`.
12. **HITL #3:** the user reviews the guide and downloads `site.zip` + `DEPLOY.md`. Memory saves their preferences.

The crews and agents

There are 4 crews with 7 agents.

Crew	Agent	Its job, simply
Planning	Product Planner	Decides WHAT pages, forms and buttons are needed, from the endpoints
Planning	UI Designer	Decides HOW it looks: layout, colors, fonts
Build	Frontend Developer	Writes the code files
Build	Code Reviewer	Checks that everything in the plan was built

Crew	Agent	Its job, simply
QA	QA Tester	Tests the pages in a real browser
QA	Bug Fixer	Fixes what the tests found
Deployment	Deployment Guide Writer	Writes step-by-step instructions to deploy the site

The crews run one after another. Each crew hands a fixed package of data to the next one: the site plan, the built files, the test report, and the deploy guide.

Guardrails and approval points

Guardrails

Guardrail	Where	What it checks
Input	Before any agent runs	It is a front-end request; each endpoint has a method and a path; secrets (tokens, passwords) are removed
Tool	When an agent writes files	Files can be written only inside the project's sandbox folder
Output	After the code is built	The code is valid, has no secrets, and loads scripts only from allowed sites
Output	After the deploy guide is written	Every step is there, and no secrets are in it

Human approval points

1. **HITL #1, approve the plan:** the user checks the pages before any code is written.

2. **HITL #2, approve the preview:** the user approves, or types a change like "make the header blue". A change goes back to the Build crew.
3. **HITL #3, review the instructions:** the user checks the deploy guide, then downloads.

Tools, MCP, knowledge and memory

Helper	Kind	Used by	Why
Web search	Tool	Product Planner (only when needed)	Look up an unfamiliar app style, e.g. "like a Kanban board"
Filesystem	MCP server	Frontend Developer, Bug Fixer	Write the code files into the sandbox folder
Playwright (optional)	MCP server	QA Tester	Open the pages in a real browser, click, and check them
UI style guide	Knowledge	UI Designer	Keep the design consistent and clean
Deployment guides	Knowledge	Deployment Guide Writer	Correct steps for GitHub Pages, Netlify, Vercel, and S3
User preferences	Memory	Read by the Planner, saved at the end	Remember choices like "likes dark themes" for next time

Example: a task manager API

A backend engineer has a Spring Boot task manager with 4 endpoints and wants a UI.

```
POST   /api/auth/login   body: {username, password}
returns: {token}
GET    /api/tasks        needs login   returns: [{id, title,
done}]
POST   /api/tasks        needs login   body: {title}
DELETE /api/tasks/{id}   needs login
```

1. **Input:** the user uploads the OpenAPI file and writes "Users log in, see their tasks, add and delete them."
2. **Input guardrail:** all 4 endpoints are valid. A test token pasted by mistake is removed, and the user is warned.
3. **Product Planner:** plans a Login page and a Tasks page with a list, an "Add task" form, a Delete button on each task, and Logout. It notes that "done" is read-only, because there is no update endpoint.
4. **UI Designer:** picks a simple layout, colors and fonts.
5. **HITL #1:** the user approves the plan.
6. **Build crew:** writes index.html, styles.css and app.js into the sandbox. The reviewer confirms all 4 endpoints are used.
7. **Preview:** the pages appear with fake tasks.
8. **QA crew:** finds that the Delete button does not refresh the list, and fixes it in round 1.
9. **HITL #2:** the user types "make the header blue". The Build crew changes it, and the user approves.
10. **Deployment crew:** the user picks Netlify and gets steps, including where to set the real API URL and how to allow it in CORS.
11. **HITL #3:** the user downloads site.zip and DEPLOY.md. Memory saves "likes blue headers".

Decisions still open

The design works with any of these answers; each one changes only a small part.

- Stacks: HTML/CSS/JS only, or also React and Vue (React and Vue can preview from a CDN with no build step)

- Use Playwright MCP for testing, or only Filesystem MCP (Playwright must be tested on Hugging Face Spaces first)
- Who owns which crew